

ARCHITECTURAL THESIS

We solve **[core user problem]** with a deliberately small, modular system that prioritizes a reliable end-to-end path, observable failures, and replaceable boundaries. The design validates the riskiest assumptions now without blocking a production-grade evolution later.

1. PROBLEM & CONSTRAINTS

- **User outcome:** [What can a user accomplish?]
- **Constraints:** 24 hours, small team, demo reliability, low operational overhead.
- **Quality bar:** Primary workflow succeeds predictably; errors are explicit and recoverable.
- **Out of scope:** [Feature, scale, or compliance deliberately deferred.]

2. COMPONENTS & DATA FLOW

USER -> CLIENT UI -> APP/API -> DATA STORE
|
+--> [EXTERNAL SERVICE]

The client handles interaction and boundary validation. The application service owns business rules and authorization. The data store is the source of truth. External integrations sit behind one adapter so they can be mocked, retried, or replaced without changing the core workflow.

3. KEY DECISIONS

[Stack]: chosen for team fluency and the fastest reliable vertical slice. A less familiar stack adds delivery risk without proving more value.

Modular monolith: one deployable unit, simple debugging, and clear module boundaries. Microservices would add networking, deployment, and consistency costs before they are justified.

[Data store]: matches the dominant access pattern and provides one source of truth. [Alternative] was weaker on [tradeoff].

4. KNOWN TRADEOFFS

- **Speed over breadth:** one complete workflow beats several fragile features.
- **Simplicity over theoretical scale:** fewer moving parts and failure surfaces today.
- **Managed services over control:** less setup, with vendor limits and cost to monitor.
- **Current uncertainty:** [Largest unvalidated assumption.]
- **Evolution trigger:** split components only when measured load, ownership, or release cadence demands it.

5. RELIABILITY & FAILURE MODES

Bad input: validate at the boundary and return actionable errors.

Dependency outage: timeout, bounded retry, fail visibly, preserve user input.

Duplicate request: stable IDs/idempotency for state changes.

Partial write: atomic operations where possible; log enough to reconcile.

Traffic spike: stateless app tier can scale; rate-limit expensive integrations.

Security: secrets stay server-side; authorize every protected operation.

6. DEFENSE UNDER Q&A

Why this architecture? It minimizes coordination and operational risk while preserving clean replacement points around data and external services.

What breaks first? [Likely bottleneck/dependency]. Detect through [logs/metrics/health check] and degrade through [fallback].

If constraints change? Add caching, queues, or replicas only after measurement; isolate high-load modules behind existing interfaces.

What would production require? Automated tests, CI/CD, monitoring and alerts, backups, threat modeling, accessibility, and load testing.

CLOSING CLAIM

This design is not finished; it is **proportionate**. It proves the riskiest user and technical assumptions with the fewest moving parts, makes failures visible, and leaves a credible path to scale.

Gauntlet AI criteria: problem, components, decisions, tradeoffs, edge cases, failure modes, and alternatives.